

Partner Webhook Security Proposal

PO Brief - Carland Service - Proposed approach for alignment

1. Executive summary

Secure multi-partner webhooks via Carland-controlled partner onboarding, one secret key per partner, and HMAC signature over the full JSON body including a Carland-assigned partnerId. Appropriate for service history data sensitivity. Stronger than a single shared secret or IP-only identification.

Recommendation: Approve as standard model for Hyper and future partners.

2. Problem today

- Single WEBHOOK_SECRET - one leak affects all partners
- Partner identity not tied to authentication - hard to scale
- recordId scoped by car only - collision risk across partners

3. Proposed model

1. Carland registers partner (manual / API / future admin UI)
2. Carland assigns partnerId + secret key
3. Partner sends JSON with partnerId + X-Signature (HMAC-SHA256, raw body)
4. Webhook gateway verifies signature with that partner secret
5. Carland ingests if partner active and payload valid

Trust rule: partnerId in body is trusted ONLY after signature verifies with that partner's secret.

4. Partner onboarding

- Now: manual DB insert or internal Postman / admin API
- Next: Admin UI - create partner, rotate secret, activate/deactivate

Each partner: partnerId, name, source/slug, active flag, secret (never re-shown in full).

5. Secret key management

Storage

- v1 (few partners): .env - WEBHOOK_SECRET_HYPER, WEBHOOK_SECRET_TOYOTA, etc.
- Scale: encrypted DB + admin UI - rotate without redeploy

Rotation

- Compromise or loss: new secret immediately; disable old after grace period
- Optional policy: weekly or monthly rotation per agreement
- Best practice: accept old + new secret for 7-14 days during rotation

6. Request format

Canonical JSON (single visit in serviceHistory) plus partnerId:

```
{ "partnerId": 1, "plate": "...", "vin": "...", "serviceHistory": [{ "recordId": ..., "services": [...] }] }
```

X-Signature = HMAC-SHA256(secret, exact raw body bytes). Compact JSON required.

Verify at webhook gateway before carland internal processing.

7. Security assessment

Protected

- Fake visits from random attackers (401 without signature)
- Body tampering (signature fails)
- Cross-partner impact from one secret leak (isolated secrets)

- Partner impersonation without victim secret

Residual risks (accepted)

- Replay - mitigated by idempotent recordId
- Stolen secret - rotation + monitoring
- DDoS - rate limits separate from auth
- Malicious insider at partner - B2B trust boundary

Conclusion: industry-standard and sufficient for service history data.

Optional v1 add-on: IP allowlist at nginx (not replacement for HMAC).

8. Comparison

- Single shared secret: Poor scale, low security
- IP whitelist only: Fragile, high ops effort
- Per-partner secret + HMAC (proposed): Good balance - RECOMMENDED
- mTLS per partner: Highest security, highest effort

9. Decisions needed from PO

- Approve per-partner HMAC model: Yes (proposed)
- partnerId in body: Required
- Secret storage v1: .env for 2-5 partners, then DB + admin UI
- Rotation schedule: Optional; mandatory on compromise
- IP allowlist: Optional per partner, not core
- Admin UI: Later - manual/API for pilots

10. Implementation phases

- Phase 1: Partner registry + partnerId in spec
- Phase 2: Per-partner secret + gateway verification
- Phase 3: DB unique (car_id, partner_id, record_id)
- Phase 4: Rotation procedure + dual-secret grace window
- Phase 5: Admin UI for partner management

11. Success criteria

- Unique partnerId and secret per active partner
- Invalid signature rejected (401)
- Invalid/inactive partnerId rejected (400/403)
- Secret rotation without affecting other partners
- One-page partner integration guide for onboarding

12. PO sign-off summary

Carland-owned partner registry; one secret per partner; partnerId in signed JSON; HMAC verification at webhook gateway. Simple for partners, scalable for Carland, adequate for service-history webhooks.

Carland Service - Partner webhook security proposal for PO review.